

NYC TLC Taxi Pipeline

Snowflake — dbt Fusion Edition

Complete start-to-finish guide with all code, exactly as shipped

Component	Pinned value
dbt engine	Fusion 2.0.0-preview.190 (FUSION_VERSION.txt)
Install method	curl script + dbt system update --version
VS Code extension	dbt (publisher: dbtLabsInc)
apache-airflow	2.9.3
Python	3.11

Every code block below is pulled directly from the shipped project files — nothing here has been retyped, so it matches what's in your zip exactly.

Part 1 — Install and pin dbt Fusion locally

1 Install the dbt VS Code extension

Publisher must be dbtLabsInc — the only one compatible with Fusion

- VS Code → Extensions tab → search 'dbt' → install the extension from dbtLabsInc
- Opening a dbt project with the extension active auto-installs the Fusion binary for you

2 Pin your local Fusion to the project's exact version

Matches Docker and CI exactly — one command

```
curl -fsSL https://public.cdn.getdbt.com/fs/install/install.sh | sh -s -- --update
dbt system update --version $(cat FUSION_VERSION.txt)
dbt --version      # confirm it matches FUSION_VERSION.txt exactly
```

FUSION_VERSION.txt (full file)

This single file is the source of truth read by your terminal, Dockerfile.airflow, and every CI/CD workflow.

```
2.0.0-preview.190
```

If you still have dbt-core installed from a previous setup, remove it: `pip uninstall dbt-core dbt-snowflake dbt-databricks -y` — otherwise it can shadow Fusion on PATH depending on shell order.

Part 2 — Unzip, workspace, and Python dependencies

1 Unzip and open the workspace

Both warehouse folders load together

```
unzip tlc_fusion.zip
cd tlc_fusion
code .
```

Double-click `tlc_fusion.code-workspace` → Open Workspace. This loads Root, Snowflake, and Databricks together with shared settings and the `dbtLabsInc` extension recommended.

2 Create the virtualenv and install pinned Python requirements

dbt itself is NOT in this file — see Part 1

```
cd snowflake
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

requirements.txt (full file)

```
# =====
# PINNED VERSIONS – used identically in your local .venv, in
# Dockerfile.airflow, and in .github/workflows/*.yml
#
# dbt itself is NOT in this file. dbt Fusion is a standalone Rust
# binary, not a Python package – it is installed via the curl
# script in Dockerfile.airflow / locally via the VS Code extension,
# and pinned with `dbt system update --version <FUSION_VERSION>`.
# See FUSION_VERSION.txt for the exact pinned build number.
# =====

# — Airflow —————
apache-airflow==2.9.3
apache-airflow-providers-snowflake==5.7.0

# — Snowflake (used by the Python loader script, NOT by dbt –
#   Fusion's Snowflake ADDBC driver downloads itself automatically) —
snowflake-connector-python==3.12.2

# — AI Agent – LangChain + Gemini (free) —————
langchain==0.2.16
langchain-core==0.2.41
langchain-google-genai==1.0.10

# Uncomment to swap to Claude API:
# langchain-anthropic==0.1.23
# anthropic==0.34.2

# Uncomment to swap to local Ollama:
# langchain-ollama==0.1.3

# — Data / ML —————
pandas==2.2.2
pyarrow==17.0.0
numpy==1.26.4
requests==2.32.3

# — Dev tools —————
python-dotenv==1.0.1
black==24.8.0
pytest==8.3.2
pre-commit==3.8.0
sqlfluff==3.1.0
```

```
sqlfluff-templater-dbt==3.1.0
```

```
python scripts/verify_versions.py
```

This parses `dbt --version` output directly and compares it against FUSION_VERSION.txt, rather than checking pip package metadata — since Fusion isn't a pip package.

scripts/verify_versions.py (full file)

```
#!/usr/bin/env python3
"""
verify_versions.py – dbt Fusion edition

Run this in ANY environment (local machine, inside the Airflow
Docker container, in CI) to confirm the dbt Fusion engine matches
the version pinned in FUSION_VERSION.txt.

Usage:
python scripts/verify_versions.py

Exit code 0 = version matches. Exit code 1 = mismatch or Fusion missing.
"""

import sys
import re
import subprocess
from pathlib import Path

def get_pinned_version():
    version_file = Path(__file__).resolve().parent.parent / "FUSION_VERSION.txt"
    if not version_file.exists():
        print(f"FAIL: FUSION_VERSION.txt not found at {version_file}")
        sys.exit(1)
    return version_file.read_text().strip()

def get_installed_version():
    try:
        result = subprocess.run(
            ["dbt", "--version"], capture_output=True, text=True, timeout=15
        )
    except FileNotFoundError:
        print("FAIL: 'dbt' command not found on PATH.")
        print("Install Fusion: curl -fsSL https://public.cdn.getdbt.com/fs/install/install.sh | sh -s -- --update")
        return None

    output = result.stdout + result.stderr

    if "Fusion" not in output and "fusion" not in output and "preview" not in output:
        # Could still be Fusion without the word appearing – fall back to regex only
        pass

    match = re.search(r"(\d+\.\d+\.\d+(?:-preview\.\d+)?)", output)
    if not match:
        print(f"FAIL: Could not parse a version number from 'dbt --version' output:\n{output}")
        return None

    return match.group(1)

def check_not_core():
    """
    Warns if dbt-core (the Python package) is also importable in this
    environment – a leftover from the old pip-based approach that could
    shadow Fusion depending on PATH order.
    """
```

```

"""
try:
    import importlib.metadata as md
    md.version("dbt-core")
    print("WARNING: dbt-core is also pip-installed in this environment.")
    print("  This project now uses dbt Fusion exclusively. Run:")
    print("  pip uninstall dbt-core dbt-snowflake dbt-databricks -y")
    return False
except Exception:
    return True

def main():
    print("=" * 60)
    print("dbt Fusion version verification")
    print("=" * 60)

    pinned = get_pinned_version()
    installed = get_installed_version()
    core_clean = check_not_core()

    if installed is None:
        print("VERSION CHECK FAILED – Fusion not found or unparseable.")
        sys.exit(1)

    print(f"Pinned (FUSION_VERSION.txt): {pinned}")
    print(f"Installed (dbt --version):   {installed}")

    if installed != pinned:
        print("\nFAIL: Fusion version mismatch.")
        print(f"  Fix locally with: dbt system update --version {pinned}")
        sys.exit(1)

    print("\nOK: Fusion version matches pin.")
    if not core_clean:
        print("(Non-fatal warning above about leftover dbt-core – clean it up when convenient.)")

    print("=" * 60)
    print("ALL CHECKS PASSED")
    sys.exit(0)

if __name__ == "__main__":
    main()

```

Part 3 — Credentials and Snowflake setup

1

Configure .env

Never committed to git

```
cp .env.example .env
# Fill in SF_ACCOUNT, SF_USER, SF_PASSWORD, GEMINI_API_KEY
```

.env.example (full file)

```
# NOTE: dbt Fusion is installed separately – see README.md and
# FUSION_VERSION.txt. It is not configured via this .env file.

# — Snowflake —————
SF_ACCOUNT=myorg-myaccount
SF_USER=your_username
SF_PASSWORD=your_password

# — AI Agent —————
GEMINI_API_KEY=AIzaSyXXXXXXXXXXXXXXXXXXXXXXX

# — n8n alerting (optional) —————
N8N_WEBHOOK_URL=http://localhost:5678/webhook/tlc-pipeline-alert
ALERT_EMAIL=your.email@gmail.com
```

2

Run the Snowflake setup SQL

Creates database, schemas, grants, stage, tables

Paste the entire file into a Snowflake worksheet and run it. It switches to ACCOUNTADMIN to grant SYSADMIN the privileges dbt needs.

scripts/setup_snowflake.sql (full file)

```
-- Run once in a Snowflake worksheet as ACCOUNTADMIN

USE ROLE ACCOUNTADMIN;

CREATE DATABASE IF NOT EXISTS TLC_DB;
CREATE WAREHOUSE IF NOT EXISTS TLC_WH
  WITH WAREHOUSE_SIZE = 'X-SMALL' AUTO_SUSPEND = 60 AUTO_RESUME = TRUE;

GRANT ALL PRIVILEGES ON DATABASE TLC_DB TO ROLE SYSADMIN;
GRANT ALL PRIVILEGES ON WAREHOUSE TLC_WH TO ROLE SYSADMIN;

USE ROLE SYSADMIN;
USE DATABASE TLC_DB;
USE WAREHOUSE TLC_WH;

CREATE SCHEMA IF NOT EXISTS TLC_DB.RAW;
CREATE SCHEMA IF NOT EXISTS TLC_DB.STAGING;
CREATE SCHEMA IF NOT EXISTS TLC_DB.MARTS;
CREATE SCHEMA IF NOT EXISTS TLC_DB.SNAPSHOTS;

USE ROLE ACCOUNTADMIN;
GRANT ALL PRIVILEGES ON SCHEMA TLC_DB.RAW TO ROLE SYSADMIN;
GRANT ALL PRIVILEGES ON SCHEMA TLC_DB.STAGING TO ROLE SYSADMIN;
GRANT ALL PRIVILEGES ON SCHEMA TLC_DB.MARTS TO ROLE SYSADMIN;
GRANT ALL PRIVILEGES ON SCHEMA TLC_DB.SNAPSHOTS TO ROLE SYSADMIN;
GRANT CREATE VIEW ON SCHEMA TLC_DB.STAGING TO ROLE SYSADMIN;
GRANT CREATE TABLE ON SCHEMA TLC_DB.MARTS TO ROLE SYSADMIN;

USE ROLE SYSADMIN;

CREATE STAGE IF NOT EXISTS TLC_DB.RAW.TLC_STAGE FILE_FORMAT = (TYPE = 'PARQUET');

CREATE OR REPLACE TABLE TLC_DB.RAW.YELLOW_TRIPS_RAW (
  VENDORID NUMBER,
  TPEP_PICKUP_DATETIME TIMESTAMP_NTZ,
  TPEP_DROPOFF_DATETIME TIMESTAMP_NTZ,
  PASSENGER_COUNT NUMBER,
  TRIP_DISTANCE FLOAT,
  RATECODEID NUMBER,
  STORE_AND_FWD_FLAG VARCHAR(1),
  PULOCATIONID NUMBER,
  DOLOCATIONID NUMBER,
  PAYMENT_TYPE NUMBER,
  FARE_AMOUNT FLOAT,
  EXTRA FLOAT,
  MTA_TAX FLOAT,
  TIP_AMOUNT FLOAT,
  TOLLS_AMOUNT FLOAT,
  IMPROVEMENT_SURCHARGE FLOAT,
  TOTAL_AMOUNT FLOAT,
  CONGESTION_SURCHARGE FLOAT,
  AIRPORT_FEE FLOAT,
  CBD_CONGESTION_FEE FLOAT,
  _LOADED_AT TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

CREATE OR REPLACE TABLE TLC_DB.RAW.TAXI_ZONE_LOOKUP (
  LOCATIONID NUMBER,
  BOROUGH VARCHAR,
  ZONE VARCHAR,
```



```
        SERVICE_ZONE VARCHAR
    );

SHOW TABLES IN SCHEMA TLC_DB.RAW;
SHOW GRANTS ON SCHEMA TLC_DB.STAGING;
```

Part 4 — Load data

1

Run the loader

Downloads Parquet, uploads to stage, COPY INTO

```
python scripts/load_to_snowflake.py --month 2024-01 --load-zones
```

scripts/load_to_snowflake.py (full file)

```
"""
load_to_snowflake.py

Downloads NYC TLC yellow taxi Parquet files and loads into Snowflake.
Includes all fixes from troubleshooting: robust .env loading, browser
User-Agent for CDN, S3 fallback, optional column handling.
"""

import os
import sys
import argparse
import logging
import requests
import snowflake.connector
from datetime import datetime, timedelta
from pathlib import Path

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

def _load_env():
    try:
        from dotenv import load_dotenv
    except ImportError:
        return
    script_dir = Path(__file__).resolve().parent
    for d in [script_dir, script_dir.parent, script_dir.parent.parent]:
        env_file = d / ".env"
        if env_file.exists():
            load_dotenv(dotenv_path=env_file, override=False)
            log.info(f"Loaded .env from: {env_file}")
            return
    log.warning(".env not found in script dir, parent, or grandparent.")

_load_env()

CDN_BASE = "https://d37ci6vzurychx.cloudfront.net/trip-data"
S3_BASE = "https://nyc-tlc.s3.amazonaws.com/trip-data"
ZONE_URL = "https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv"
ZONE_S3 = "https://nyc-tlc.s3.amazonaws.com/misc/taxi+zone_lookup.csv"
TMP_DIR = Path("/tmp/tlc")
TMP_DIR.mkdir(parents=True, exist_ok=True)

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
        "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
    ),
    "Accept": "application/octet-stream,*/*",
}

LATEST_AVAILABLE = "2025-03"

def get_target_month(month_str=None):
    if month_str:
        return month_str
    log.warning(f"No --month given. Defaulting to {LATEST_AVAILABLE}.")
    return LATEST_AVAILABLE
```

```

def download_file(url, dest, fallback_url=None):
    for attempt in ([url] + ([fallback_url] if fallback_url else [])):
        log.info(f"Downloading {attempt}")
        try:
            with requests.get(attempt, stream=True, timeout=180, headers=HEADERS) as r:
                if r.status_code in (403, 404):
                    log.warning(f"{r.status_code} on {attempt} - trying fallback...")
                    continue
                r.raise_for_status()
                with open(dest, "wb") as f:
                    for chunk in r.iter_content(65536):
                        f.write(chunk)
                log.info(f"Saved {Path(dest).stat().st_size/1_000_000:.1f} MB -> {dest}")
                return dest
        except requests.exceptions.HTTPError as e:
            log.warning(f"HTTP error: {e}")
            continue
    raise RuntimeError(f"Could not download from {url} or {fallback_url}")

def get_snowflake_conn():
    missing = [v for v in ["SF_ACCOUNT", "SF_USER", "SF_PASSWORD"] if not os.environ.get(v)]
    if missing:
        log.error(f"Missing env vars: {missing}")
        sys.exit(1)
    return snowflake.connector.connect(
        account=os.environ["SF_ACCOUNT"],
        user=os.environ["SF_USER"],
        password=os.environ["SF_PASSWORD"],
        database=os.environ.get("SF_DATABASE", "TLC_DB"),
        warehouse=os.environ.get("SF_WAREHOUSE", "TLC_WH"),
        schema="RAW",
        role=os.environ.get("SF_ROLE", "SYSADMIN"),
    )

def load_trips(month, conn):
    cdn_url = f"{CDN_BASE}/yellow_tripdata_{month}.parquet"
    s3_url = f"{S3_BASE}/yellow_tripdata_{month}.parquet"
    fpath = TMP_DIR / f"yellow_tripdata_{month}.parquet"
    download_file(cdn_url, fpath, fallback_url=s3_url)

    cur = conn.cursor()
    try:
        cur.execute(f"PUT file://{fpath} @TLC_DB.RAW.TLC_STAGE AUTO_COMPRESS=FALSE OVERWRITE=TRUE")
        cur.execute(f"""
            COPY INTO TLC_DB.RAW.YELLOW_TRIPS_RAW
            FROM @TLC_DB.RAW.TLC_STAGE/yellow_tripdata_{month}.parquet
            FILE_FORMAT = (TYPE='PARQUET' SNAPPY_COMPRESSION=TRUE)
            MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
            ON_ERROR = 'CONTINUE'
            PURGE = FALSE;
        """)
        log.info(f"COPY INTO result: {cur.fetchall()}")
    finally:
        cur.close()

def load_zones(conn):
    fpath = TMP_DIR / "taxi_zone_lookup.csv"
    download_file(ZONE_URL, fpath, fallback_url=ZONE_S3)
    cur = conn.cursor()
    try:
        cur.execute(f"PUT file://{fpath} @TLC_DB.RAW.TLC_STAGE AUTO_COMPRESS=FALSE OVERWRITE=TRUE")
        cur.execute(f"""
            COPY INTO TLC_DB.RAW.TAXI_ZONE_LOOKUP
            FROM @TLC_DB.RAW.TLC_STAGE/taxi_zone_lookup.csv
            FILE_FORMAT = (TYPE='CSV' FIELD_OPTIONALLY_ENCLOSED_BY='"' SKIP_HEADER=1)
            ON_ERROR = 'CONTINUE'
            PURGE = FALSE;
        """)
    finally:
        cur.close()

```

```
        log.info("Zone lookup loaded.")
    finally:
        cur.close()

def main(month=None, load_zones_flag=False):
    target = get_target_month(month)
    log.info(f"Target month: {target}")
    conn = get_snowflake_conn()
    try:
        load_trips(target, conn)
        if load_zones_flag:
            load_zones(conn)
    finally:
        conn.close()
    log.info("Done.")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--month", type=str, default=None)
    parser.add_argument("--load-zones", action="store_true", default=False)
    args = parser.parse_args()
    main(args.month, args.load_zones)
```

Part 5 — dbt project (Fusion syntax)

1

Install dbt packages and verify connection

dbt deps / dbt debug work identically to Core — Fusion is Jinja-compatible

```
cd dbt
dbt deps --profiles-dir .
dbt debug --profiles-dir .
```

dbt_project.yml

require-dbt-version is now `>=2.0.0,<3.0.0` — this signals Fusion compatibility and refuses to run under dbt Core v1.

```
name: 'tlc_snowflake'
version: '1.0.0'
config-version: 2

profile: 'tlc_snowflake'

# — PINS THE PROJECT TO dbt Fusion (2.0.0+) —————
# Signals this project targets Fusion, not dbt Core v1. If someone
# opens it with dbt-core 1.x, they'll get a clear version warning
# instead of confusing silent behaviour differences.
require-dbt-version: [">>=2.0.0", "<3.0.0"]

model-paths: ["models"]
test-paths: ["tests"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

target-path: "target"
clean-targets: ["target", "dbt_packages"]

models:
  tlc_snowflake:
    staging:
      +materialized: view
    marts:
      +materialized: table
    ai:
      +materialized: table

vars:
  min_fare: 0.5
  max_fare: 500
  max_distance: 100
```

profiles.yml

```
tlc_snowflake:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "{{ env_var('SF_ACCOUNT') }}"
      user: "{{ env_var('SF_USER') }}"
      password: "{{ env_var('SF_PASSWORD') }}"
      role: SYSADMIN
      database: TLC_DB
      warehouse: TLC_WH
      schema: STAGING
      threads: 4

  prod:
    type: snowflake
```

```
account: "{{ env_var('SF_ACCOUNT') }}"
user:    "{{ env_var('SF_USER') }}"
password: "{{ env_var('SF_PASSWORD') }}"
role:    SYSADMIN
database: TLC_DB
warehouse: TLC_WH
schema:  STAGING
threads: 8
```

packages.yml

```
packages:
- package: dbt-labs/dbt_utils
  version: [">>=1.3.0", "<1.4.0"]
```

Staging models

models/staging/stg_yellow_trips.sql

```
-- models/staging/stg_yellow_trips.sql
with source as (
  select * from {{ source('raw', 'yellow_trips_raw') }}
),

cleaned as (
  select
    tpep_pickup_datetime                as pickup_ts,
    tpep_dropoff_datetime              as dropoff_ts,
    datediff('minute', tpep_pickup_datetime, tpep_dropoff_datetime) as trip_duration_mins,
    trip_distance,
    passenger_count,
    pulocationid,
    dolocationid,
    ratecodeid,
    store_and_fwd_flag,
    fare_amount,
    extra,
    mta_tax,
    tip_amount,
    tolls_amount,
    improvement_surcharge,
    congestion_surcharge,
    coalesce(airport_fee, 0)           as airport_fee,
    coalesce(cbd_congestion_fee, 0) as cbd_congestion_fee,
    total_amount,
    payment_type,
    vendorid,
    current_timestamp()               as _dbt_loaded_at
  from source
  where fare_amount > {{ var('min_fare') }}
    and trip_distance > 0
    and passenger_count between 1 and 6
    and datediff('minute', tpep_pickup_datetime, tpep_dropoff_datetime) between 1 and 180
    and tpep_pickup_datetime is not null
    and tpep_dropoff_datetime is not null
)

select * from cleaned
```

models/staging/stg_zone_lookup.sql

```
-- models/staging/stg_zone_lookup.sql
select
  locationid as location_id,
  borough,
  zone,
  service_zone
from {{ source('raw', 'taxi_zone_lookup') }}
where locationid is not null
```

Mart models

models/marts/dim_zones.sql

```
-- models/marts/dim_zones.sql
select location_id, borough, zone, service_zone
from {{ ref('stg_zone_lookup') }}
```

models/marts/fct_trips.sql — standard full-rebuild version

```
-- models/marts/fct_trips.sql
-- Standard full-rebuild fact table. For the incremental version
-- used in production, see fct_trips_incremental.sql

with trips as (
    select * from {{ ref('stg_yellow_trips') }}
),
pickup_zones as (
    select location_id, borough as pickup_borough, zone as pickup_zone, service_zone as
pickup_service_zone
    from {{ ref('dim_zones') }}
),
dropoff_zones as (
    select location_id, borough as dropoff_borough, zone as dropoff_zone
    from {{ ref('dim_zones') }}
)

select
    t.pickup_ts,
    t.dropoff_ts,
    t.trip_duration_mins,
    date_trunc('day', t.pickup_ts) as pickup_date,
    date_trunc('hour', t.pickup_ts) as pickup_hour,
    hour(t.pickup_ts) as pickup_hour_of_day,
    dayofweek(t.pickup_ts) as pickup_dow,

    t.pulocationid,
    t.dolocationid,
    pu.pickup_borough,
    pu.pickup_zone,
    pu.pickup_service_zone,
    do.dropoff_borough,
    do.dropoff_zone,

    t.trip_distance,
    t.passenger_count,
    t.ratecodeid,

    t.fare_amount,
    t.tip_amount,
    t.tolls_amount,
    t.congestion_surcharge,
    t.airport_fee,
    t.cbd_congestion_fee,
    t.total_amount,
    t.payment_type,

    case t.payment_type
        when 1 then 'Credit card'
        when 2 then 'Cash'
        when 3 then 'No charge'
        when 4 then 'Dispute'
        when 5 then 'Unknown'
        when 6 then 'Voided'
        else 'Other'
    end as payment_type_desc,

    round(t.tip_amount / nullif(t.fare_amount, 0) * 100, 2) as tip_pct,

    t.vendorid,
```



```
t._dbt_loaded_at
```

```
from trips t
left join pickup_zones pu on t.pulocationid = pu.location_id
left join dropoff_zones do on t.dolocationid = do.location_id
```

models/marts/fct_trips_incremental.sql — production version

```
-- models/marts/fct_trips_incremental.sql
-- Incremental version – only processes rows newer than what's
-- already in the table. Use this one in production; fct_trips.sql
-- is kept as the simple full-rebuild reference version.

{{ config(
    materialized      = 'incremental',
    unique_key        = ['pickup_ts', 'pulocationid', 'dolocationid', 'fare_amount'],
    on_schema_change  = 'sync_all_columns'
) }}

with trips as (
    select * from {{ ref('stg_yellow_trips') }}

    {% if is_incremental() %}
        where pickup_ts > (select max(pickup_ts) from {{ this }})
    {% endif %}
),
pickup_zones as (
    select location_id, borough as pickup_borough, zone as pickup_zone, service_zone as
pickup_service_zone
    from {{ ref('dim_zones') }}
),
dropoff_zones as (
    select location_id, borough as dropoff_borough, zone as dropoff_zone
    from {{ ref('dim_zones') }}
)

select
    t.pickup_ts,
    t.dropoff_ts,
    t.trip_duration_mins,
    date_trunc('day', t.pickup_ts) as pickup_date,
    date_trunc('hour', t.pickup_ts) as pickup_hour,
    hour(t.pickup_ts) as pickup_hour_of_day,
    dayofweek(t.pickup_ts) as pickup_dow,

    t.pulocationid,
    t.dolocationid,
    pu.pickup_borough,
    pu.pickup_zone,
    pu.pickup_service_zone,
    do.dropoff_borough,
    do.dropoff_zone,

    t.trip_distance,
    t.passenger_count,
    t.ratecodeid,

    t.fare_amount,
    t.tip_amount,
    t.tolls_amount,
    t.congestion_surcharge,
    t.airport_fee,
    t.cbd_congestion_fee,
    t.total_amount,
    t.payment_type,

    case t.payment_type
        when 1 then 'Credit card'
        when 2 then 'Cash'
        when 3 then 'No charge'
        when 4 then 'Dispute'
        when 5 then 'Unknown'
        when 6 then 'Voided'
        else 'Other'
    end as payment_type_desc,

    round(t.tip_amount / nullif(t.fare_amount, 0) * 100, 2) as tip_pct,

    t.vendorid,
```

```
    t._dbt_loaded_at,  
    current_timestamp() as _processed_at  
from trips t  
left join pickup_zones pu on t.pulocationid = pu.location_id  
left join dropoff_zones do on t.dolocationid = do.location_id
```

models/marts/agg_daily_revenue.sql

```
-- models/marts/agg_daily_revenue.sql
select
  pickup_date as trip_date,
  pickup_borough,
  count(*) as total_trips,
  round(sum(total_amount), 2) as total_revenue,
  round(avg(fare_amount), 2) as avg_fare,
  round(avg(tip_amount), 2) as avg_tip,
  round(avg(trip_distance), 2) as avg_distance_miles,
  round(avg(trip_duration_mins), 2) as avg_duration_mins,
  countif(payment_type = 1) as card_trips,
  countif(payment_type = 2) as cash_trips
from {{ ref('fct_trips') }}
group by 1, 2
```

models/marts/agg_hourly_demand.sql

```
-- models/marts/agg_hourly_demand.sql
select
  pickup_hour_of_day,
  pickup_dow,
  pickup_borough,
  count(*) as total_trips,
  round(avg(fare_amount), 2) as avg_fare
from {{ ref('fct_trips') }}
group by 1, 2, 3
```

AI anomaly detection

Pure SQL z-score model. Fusion's support for Python dbt models is limited, so avoiding them entirely (as this project already did) sidesteps that question completely.

models/ai/anomaly_flags.sql

```
-- models/ai/anomaly_flags.sql
-- Pure SQL anomaly detection using z-scores per borough.
-- Avoids Snowpark Python models entirely – no special permissions
-- needed, runs faster, and is easier to explain in an interview
-- than a black-box Isolation Forest call.

with trips as (
  select * from {{ ref('fct_trips') }}
),

stats as (
  select
    pickup_borough,
    avg(fare_amount)           as mean_fare,
    stddev(fare_amount)        as std_fare,
    avg(trip_distance)         as mean_distance,
    stddev(trip_distance)      as std_distance,
    avg(trip_duration_mins)    as mean_duration,
    stddev(trip_duration_mins) as std_duration
  from trips
  group by pickup_borough
),

scored as (
  select
    t.pickup_ts,
    t.fare_amount,
    t.trip_distance,
    t.trip_duration_mins,
    t.passenger_count,
    t.pickup_borough,
    t.payment_type_desc,

    abs(t.fare_amount      - s.mean_fare) / nullif(s.std_fare, 0)   as fare_zscore,
    abs(t.trip_distance    - s.mean_distance) / nullif(s.std_distance, 0) as distance_zscore,
    abs(t.trip_duration_mins - s.mean_duration) / nullif(s.std_duration, 0) as duration_zscore

  from trips t
  left join stats s on t.pickup_borough = s.pickup_borough
),

flagged as (
  select
    pickup_ts,
    fare_amount,
    trip_distance,
    trip_duration_mins,
    passenger_count,
    pickup_borough,
    payment_type_desc,
    fare_zscore,
    distance_zscore,
    duration_zscore,

    round(
      (coalesce(fare_zscore, 0) + coalesce(distance_zscore, 0) + coalesce(duration_zscore, 0)) /
      3,
      2
    ) as anomaly_score,

    case
      when fare_zscore      > 3 then true
      when distance_zscore > 3 then true
      when duration_zscore > 3 then true
      else false
    end as anomaly_flag
  from scored
)
```

```
        end as is_anomaly,

        case
            when fare_zscore > 3 and distance_zscore > 3 then 'high fare + long distance'
            when fare_zscore > 3 and duration_zscore > 3 then 'high fare + long duration'
            when fare_zscore > 3 then 'high fare'
            when distance_zscore > 3 then 'long distance'
            when duration_zscore > 3 then 'long duration'
            else null
        end as anomaly_reason

    from scored
)

select * from flagged
```

Tests, snapshots, and schema — Fusion syntax

tests/assert_no_future_trips.sql — data contract test

```
-- tests/assert_no_future_trips.sql
-- Data contract: fails the run if any pickup timestamps are in
-- the future, which would indicate a timezone or data quality bug
-- upstream. Returns 0 rows when passing.

select
  pickup_ts,
  fare_amount,
  pulocationid
from {{ ref('fct_trips') }}
where pickup_ts > current_timestamp()
```

snapshots/zone_snapshot.sql — SCD Type 2

```
-- snapshots/zone_snapshot.sql
{% snapshot zone_snapshot %}
{{
  config(
    target_schema = 'snapshots',
    unique_key    = 'location_id',
    strategy      = 'check',
    check_cols    = ['borough', 'zone', 'service_zone']
  )
}}

select
  location_id,
  borough,
  zone,
  service_zone
from {{ ref('dim_zones') }}

{% endsnapshot %}
```

models/schema.yml — generic test arguments nested under 'arguments:'

This is the exact syntax Fusion requires and enforces more strictly than dbt Core — every `accepted_range` and `accepted_values` test below nests its parameters under an `arguments:` key.

```
version: 2

# — Syntax confirmed for dbt Fusion (2.0.0-preview.190) ———
# Fusion requires generic test parameters nested under an
# 'arguments:' key — this is the exact syntax Fusion enforces
# more strictly than dbt Core did. data_tests: (not tests:) is
# required in both engines as of 1.8+.

sources:
  - name: raw
    database: TLC_DB
    schema: RAW
    description: "Raw tables loaded directly from TLC Parquet files via Airflow"
    freshness:
      warn_after: {count: 49, period: hour}
      error_after: {count: 73, period: hour}
    loaded_at_field: _loaded_at
    tables:
      - name: yellow_trips_raw
        description: "Raw yellow taxi trip records from TLC public dataset"
      - name: taxi_zone_lookup
        description: "TLC LocationID to borough and zone name mapping"
        columns:
          - name: LOCATIONID
            data_tests:
              - unique
              - not_null

models:
  - name: stg_yellow_trips
    description: "Cleaned and typed yellow taxi trips from RAW layer"
    columns:
      - name: pickup_ts
        data_tests:
          - not_null
      - name: dropoff_ts
        data_tests:
          - not_null
      - name: trip_duration_mins
        data_tests:
          - not_null
          - dbt_utils.accepted_range:
              arguments:
                min_value: 1
                max_value: 180
              config:
                severity: warn
      - name: fare_amount
        data_tests:
          - not_null
          - dbt_utils.accepted_range:
              arguments:
                min_value: 0.5
                max_value: 500
      - name: trip_distance
        data_tests:
          - not_null
          - dbt_utils.accepted_range:
              arguments:
                min_value: 0
                max_value: 100
      - name: payment_type
        data_tests:
          - accepted_values:
              arguments:
                values: [1, 2, 3, 4, 5, 6]

  - name: dim_zones
    description: "Zone dimension table with borough and neighbourhood"
```



```

columns:
  - name: location_id
    data_tests:
      - unique
      - not_null

- name: fct_trips
  description: "Core fact table joining trips with zone names"
  columns:
    - name: pickup_ts
      data_tests:
        - not_null
    - name: fare_amount
      data_tests:
        - not_null
    - name: pickup_borough
      data_tests:
        - not_null

- name: fct_trips_incremental
  description: "Incremental version of fct_trips – only processes new rows"
  columns:
    - name: pickup_ts
      data_tests:
        - not_null

- name: agg_daily_revenue
  description: "Daily revenue and trip counts aggregated by borough"
  columns:
    - name: trip_date
      data_tests:
        - not_null
    - name: total_revenue
      data_tests:
        - not_null

- name: agg_hourly_demand
  description: "Trip count by hour of day and borough"

- name: anomaly_flags
  description: "Z-score based anomaly detection on fare, distance, and duration"
  columns:
    - name: pickup_ts
      data_tests:
        - not_null
    - name: is_anomaly
      data_tests:
        - accepted_values:
            arguments:
              values: [true, false]

```

2

Run the full dbt project

Builds, tests, snapshots, docs — commands are identical to dbt Core

```
dbt run      --profiles-dir .
dbt test     --profiles-dir .
dbt snapshot --profiles-dir .
dbt docs generate --profiles-dir . && dbt docs serve --profiles-dir .
```

Part 6 — Airflow with the Fusion-pinned Docker image

The custom Dockerfile runs the same curl install and version-pin command your terminal ran in Part 1, then fails the build outright if the resulting version doesn't match FUSION_VERSION.txt.

Dockerfile.airflow

```
# =====
# Dockerfile.airflow – Snowflake variant, dbt Fusion edition
#
# Installs the dbt Fusion engine (standalone Rust binary) inside
# the Airflow container at the EXACT SAME version pinned in
# FUSION_VERSION.txt – the same file you use to set your local
# VS Code extension version. One file, one version, both places.
# =====

FROM apache/airflow:2.9.3-python3.11

USER root

# curl is needed to run the Fusion installer; not present in the
# slim base image by default
RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates \
    && rm -rf /var/lib/apt/lists/*

USER airflow

# Install pinned Python dependencies (Airflow providers, loader deps,
# AI agent – NOT dbt, which is installed separately below)
COPY requirements.txt /requirements.txt
RUN pip install --no-cache-dir --user -r /requirements.txt

# — Install dbt Fusion engine, pinned to FUSION_VERSION.txt —
COPY FUSION_VERSION.txt /FUSION_VERSION.txt

RUN curl -fsSL https://public.cdn.getdbt.com/fs/install/install.sh | sh -s -- --update \
    && export PATH="$HOME/.local/bin:$PATH" \
    && FUSION_VERSION=$(cat /FUSION_VERSION.txt) \
    && dbt system update --version "$FUSION_VERSION" \
    && dbt --version

# Make sure the Fusion binary is on PATH for every shell in the container
ENV PATH="/home/airflow/.local/bin:${PATH}"

# Fail the build immediately if the installed version doesn't match
# the pin – catches drift before any DAG ever runs, not after
RUN FUSION_VERSION=$(cat /FUSION_VERSION.txt) \
    && INSTALLED=$(dbt --version 2>&1 | grep -oE '[0-9]+\.[0-9]+\.[0-9]+(-preview\.[0-9]+)?' | head -
1) \
    && echo "Pinned: $FUSION_VERSION | Installed: $INSTALLED" \
    && if [ "$INSTALLED" != "$FUSION_VERSION" ]; then \
        echo "FUSION VERSION MISMATCH – build aborted" && exit 1; \
    fi

USER root
```

docker-compose.yml

```
version: '3.8'

x-airflow-common: &airflow-common
  build:
    context: .
    dockerfile: Dockerfile.airflow
  platform: linux/amd64
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: 'true'
    PYTHONPATH: /home/airflow/.local/lib/python3.11/site-packages
    SF_ACCOUNT: ${SF_ACCOUNT}
    SF_USER: ${SF_USER}
    SF_PASSWORD: ${SF_PASSWORD}
    SF_DATABASE: TLC_DB
    SF_WAREHOUSE: TLC_WH
    SF_ROLE: SYSADMIN
    GEMINI_API_KEY: ${GEMINI_API_KEY}
    N8N_WEBHOOK_URL: ${N8N_WEBHOOK_URL}
  volumes:
    - ./airflow/dags:/opt/airflow/dags
    - ./scripts:/opt/airflow/scripts
    - ./dbt:/opt/airflow/dbt
    - ./logs:/opt/airflow/logs
  depends_on:
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
      start_period: 5s

  airflow-webserver:
    <<: *airflow-common
    command: webserver
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s

  airflow-scheduler:
    <<: *airflow-common
    command: scheduler

  airflow-init:
    <<: *airflow-common
    entrypoint: /bin/bash
    command:
      - -c
      - |
        airflow db init
        airflow users create \
          --username admin --password admin \
          --firstname Admin --lastname User \
          --role Admin --email admin@example.com
```

```
volumes:  
  postgres-db-volume:
```

airflow/dags/tlc_snowflake_dag.py

```
"""
tlc_snowflake_dag.py

Full pipeline DAG. Runs inside the custom Docker image built from
Dockerfile.airflow, which has dbt-core==1.10.19 + dbt-snowflake==1.11.1
pre-installed – the exact same versions as your local .env.
"""

from __future__ import annotations
import os, sys, requests, logging
from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.bash import BashOperator
from airflow.utils.trigger_rule import TriggerRule

log = logging.getLogger(__name__)

DBT_PROJECT_DIR = "/opt/airflow/dbt"
N8N_WEBHOOK = os.environ.get("N8N_WEBHOOK_URL", "")
TLC_BASE = "https://d37ci6vzurychx.cloudfront.net/trip-data"

DEFAULT_ARGS = {
    "owner": "data-engineering",
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
    "email_on_failure": False,
}

def _get_target_month(**context):
    logical_date = context["logical_date"]
    first = logical_date.replace(day=1)
    two_back = (first - timedelta(days=32)).replace(day=1)
    month = two_back.strftime("%Y-%m")
    context["ti"].xcom_push(key="target_month", value=month)
    return month

def _check_source(**context):
    month = context["ti"].xcom_pull(key="target_month", task_ids="get_target_month")
    url = f"{TLC_BASE}/yellow_tripdata_{month}.parquet"
    resp = requests.head(url, timeout=30, headers={"User-Agent": "Mozilla/5.0"})
    return "load_to_snowflake" if resp.status_code == 200 else "notify_failure"

def _load_to_snowflake(**context):
    month = context["ti"].xcom_pull(key="target_month", task_ids="get_target_month")
    sys.path.insert(0, "/opt/airflow/scripts")
    from load_to_snowflake import main
    main(month=month, load_zones_flag=False)

def _notify(status, **context):
    if not N8N_WEBHOOK:
        return
    month = context["ti"].xcom_pull(key="target_month", task_ids="get_target_month")
    try:
        requests.post(N8N_WEBHOOK, json={
            "status": status, "month": month,
            "dag_id": context["dag"].dag_id, "run_id": context["run_id"],
            "timestamp": datetime.utcnow().isoformat(),
        }, timeout=10)
    except Exception as e:
        log.warning(f"n8n notify failed: {e}")

with DAG(
```

```

dag_id="tlc_snowflake_pipeline",
default_args=DEFAULT_ARGS,
description="NYC TLC -> Snowflake -> dbt monthly pipeline (version-locked)",
schedule="0 6 1 * *",
start_date=datetime(2024, 1, 1),
catchup=False,
max_active_runs=1,
tags=["tlc", "snowflake", "dbt", "portfolio"],
) as dag:

    get_target_month = PythonOperator(task_id="get_target_month", python_callable=_get_target_month)
    check_source = BranchPythonOperator(task_id="check_source", python_callable=_check_source)
    load_to_snowflake = PythonOperator(task_id="load_to_snowflake",
python_callable=_load_to_snowflake)

    verify_versions = BashOperator(
        task_id="verify_versions",
        bash_command="python /opt/airflow/scripts/verify_versions.py",
    )

    dbt_deps = BashOperator(
        task_id="dbt_deps",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt deps --profiles-dir .",
    )
    dbt_run = BashOperator(
        task_id="dbt_run",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt run --profiles-dir . --target prod",
    )
    dbt_test = BashOperator(
        task_id="dbt_test",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt test --profiles-dir . --target prod",
    )
    dbt_snapshot = BashOperator(
        task_id="dbt_snapshot",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt snapshot --profiles-dir . --target prod",
    )
    dbt_docs = BashOperator(
        task_id="dbt_docs_generate",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt docs generate --profiles-dir . --target prod",
    )

    notify_success = PythonOperator(
        task_id="notify_success", python_callable=_notify,
        op_kwargs={"status": "success"}, trigger_rule=TriggerRule.ALL_SUCCESS,
    )
    notify_failure = PythonOperator(
        task_id="notify_failure", python_callable=_notify,
        op_kwargs={"status": "failure"}, trigger_rule=TriggerRule.ONE_FAILED,
    )

    (
        get_target_month
        >> check_source
        >> load_to_snowflake
        >> verify_versions
        >> dbt_deps
        >> dbt_run
        >> dbt_test
        >> dbt_snapshot
        >> dbt_docs
        >> notify_success
    )
    [check_source, load_to_snowflake, verify_versions, dbt_deps, dbt_run, dbt_test] >> notify_failure

```

1

Build and start Airflow

--build forces the Fusion install layer to actually re-run

```
docker-compose up -d --build  
# Open: http://localhost:8080 admin/admin
```

2

Trigger and watch the DAG

verify_versions runs before any dbt command

- Unpause `tlc_snowflake_pipeline`
- Click the play button → Trigger DAG
- Task order: `get_target_month` → `check_source` → `load_to_snowflake` → `verify_versions` → `dbt_deps` → `dbt_run` → `dbt_test` → `dbt_snapshot` → `dbt_docs_generate` → `notify_success`
- If `verify_versions` fails, every downstream task is skipped — check that task's logs first

Part 7 — AI agent

Uses Gemini 1.5 Flash (free) by default. Calls `verify_versions` as its first tool before touching dbt at all.

agent/pipeline_agent.py

```
"""
pipeline_agent.py

Agentic pipeline runner using LangChain + Gemini 1.5 Flash (free).
Works for both Snowflake and Databricks – pass --warehouse.

Usage:
  python agent/pipeline_agent.py --warehouse snowflake --month 2024-01
  python agent/pipeline_agent.py --warehouse databricks --month 2024-01
  python agent/pipeline_agent.py --warehouse snowflake --dry-run
"""

import os
import sys
import subprocess
import argparse
import json
import logging
from datetime import datetime
from pathlib import Path
from dotenv import load_dotenv

for d in [Path(__file__).parent, Path(__file__).parent.parent]:
    if (d / ".env").exists():
        load_dotenv(d / ".env")
        break

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

PROJECT_ROOT = Path(__file__).resolve().parent.parent

def get_llm():
    # OPTION 1: Gemini 1.5 Flash – FREE
    from langchain_google_genai import ChatGoogleGenerativeAI
    return ChatGoogleGenerativeAI(
        model="gemini-1.5-flash",
        temperature=0,
        google_api_key=os.environ["GEMINI_API_KEY"],
    )
    # OPTION 2: Ollama – uncomment for fully offline
    # from langchain_ollama import ChatOllama
    # return ChatOllama(model="llama3", temperature=0)
    # OPTION 3: Claude Haiku – uncomment for best quality
    # from langchain_anthropic import ChatAnthropic
    # return ChatAnthropic(model="claude-haiku-4-5-20251001", temperature=0,
    #     api_key=os.environ["ANTHROPIC_API_KEY"])

def run_shell(cmd, cwd=None):
    log.info(f"$ {cmd}")
    result = subprocess.run(cmd, shell=True, capture_output=True, text=True, cwd=cwd or
str(PROJECT_ROOT))
    return {
        "stdout": result.stdout[-3000:] if result.stdout else "",
        "stderr": result.stderr[-1500:] if result.stderr else "",
        "returncode": result.returncode,
        "success": result.returncode == 0,
    }

def build_tools():
    dbt_dir = str(PROJECT_ROOT / "dbt")
    script_dir = str(PROJECT_ROOT / "scripts")
```

```

    return {
        "verify_versions": {
            "description": "Confirm dbt-core version matches the pinned requirements (catches Fusion-
vs-Core drift)",
            "parameters": {},
            "fn": lambda: run_shell(f"python {script_dir}/verify_versions.py"),
        },
        "download_and_load": {
            "description": "Download TLC parquet for a month and load into the warehouse",
            "parameters": {"month": "YYYY-MM string"},
            "fn": None, # set in run_agent() based on --warehouse
        },
        "dbt_run": {
            "description": "Run all dbt models",
            "parameters": {"select": "optional model/tag, default 'all'"},
            "fn": lambda select="all": run_shell(
                f"cd {dbt_dir} && dbt run --profiles-dir ." + (f" --select {select}" if select !=
"all" else "")
            ),
        },
        "dbt_test": {
            "description": "Run dbt data quality tests",
            "parameters": {},
            "fn": lambda: run_shell(f"cd {dbt_dir} && dbt test --profiles-dir ."),
        },
        "dbt_snapshot": {
            "description": "Run dbt snapshots (SCD Type 2 zone tracking)",
            "parameters": {},
            "fn": lambda: run_shell(f"cd {dbt_dir} && dbt snapshot --profiles-dir ."),
        },
        "dbt_docs": {
            "description": "Generate dbt lineage documentation",
            "parameters": {},
            "fn": lambda: run_shell(f"cd {dbt_dir} && dbt docs generate --profiles-dir ."),
        },
    }
}

```

SYSTEM_PROMPT = """You are a data pipeline orchestration agent for the NYC TLC taxi pipeline.
Warehouse: {warehouse}

Available tools: {tools}

Always call verify_versions FIRST before anything else – if dbt versions don't match the pinned requirements, stop and report it rather than proceeding, since results would be unreliable.

Pipeline order after version check:

1. download_and_load
2. dbt_run
3. dbt_test
4. dbt_snapshot
5. dbt_docs

Step filter: {step}

Dry run: {dry_run}

Respond ONLY with valid JSON:

```

{{"action": "tool_name", "parameters": {{...}}}}
{{"action": "done", "summary": "..."}}

```

```

def run_agent(warehouse, month, step="all", dry_run=False):
    llm = get_llm()
    tools = build_tools()
    # Fix the loader command for the specific warehouse
    loader_script = "load_to_snowflake.py" if warehouse == "snowflake" else "load_to_databricks.py"
    tools["download_and_load"]["fn"] = lambda month: run_shell(
        f"python {PROJECT_ROOT}/scripts/{loader_script} --month {month}"
    )

```

```

tools_desc = json.dumps({n: {"description": t["description"], "parameters": t["parameters"]} for
n, t in tools.items()}, indent=2)
task = f"Run the TLC pipeline.\nMonth: {month}\nStep: {step}\nDry run: {dry_run}\nTime:
{datetime.utcnow().isoformat()}"

from langchain_core.messages import HumanMessage, SystemMessage, AIMessage
system = SYSTEM_PROMPT.format(warehouse=warehouse, tools=tools_desc, step=step, dry_run=dry_run)
history = [SystemMessage(content=system), HumanMessage(content=task)]
results = []

log.info(f"Agent starting - warehouse={warehouse}, month={month}")

for i in range(14):
    response = llm.invoke(history)
    content = response.content.strip()
    if content.startswith("`"):
        content = "\n".join(l for l in content.split("\n") if not
l.strip().startswith("`")).strip()
    try:
        decision = json.loads(content)
    except json.JSONDecodeError:
        history.append(AIMessage(content=content))
        history.append(HumanMessage(content="Respond only with valid JSON."))
        continue

    history.append(AIMessage(content=json.dumps(decision)))

    if decision.get("action") == "done":
        summary = decision.get("summary", "Pipeline complete.")
        print(f"\n{'='*60}\nAGENT SUMMARY - {warehouse.upper()}\n{'='*60}\n{summary}\n{'='*60}")
        return {"status": "success", "summary": summary, "log": results}

    tool_name = decision.get("action")
    params = decision.get("parameters", {})
    if tool_name not in tools:
        history.append(HumanMessage(content=f"Unknown tool '{tool_name}'. Available:
{list(tools.keys())}"))
        continue

    if dry_run:
        result = {"stdout": f"[DRY RUN] {tool_name}({params})", "success": True, "returncode": 0,
"stderr": ""}
    else:
        result = tools[tool_name]["fn"](**params)

    results.append({"tool": tool_name, "params": params, "result": result})
    log.info(f" -> {'OK' if result['success'] else 'FAILED'}")

    feedback = json.dumps({"tool": tool_name, "success": result["success"], "stdout":
result["stdout"][:2000], "stderr": result["stderr"][:500]})
    history.append(HumanMessage(content=f"Tool result: {feedback}"))

    return {"status": "incomplete", "log": results}

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--warehouse", choices=["snowflake", "databricks"], required=True)
    parser.add_argument("--month", type=str, default="2024-03")
    parser.add_argument("--step", type=str, default="all")
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()
    run_agent(args.warehouse, args.month, args.step, args.dry_run)

```

1

Get a free Gemini key and run the agent

aistudio.google.com/app/apikey

```
python agent/pipeline_agent.py --warehouse snowflake --month 2024-02 --dry-run
python agent/pipeline_agent.py --warehouse snowflake --month 2024-02
```

Part 8 — n8n Gmail alerts

1

Start n8n and import the workflow

Docker, free, self-hosted

```
docker run -it --rm --name n8n -p 5678:5678 -v ~/.n8n:/home/node/.n8n n8nio/n8n
# Open: http://localhost:5678
# Import n8n/n8n_gmail_workflow.json
```

Connect Gmail via OAuth2 in either Gmail node. Set ALERT_EMAIL in n8n's environment variables. Copy the webhook URL into .env as N8N_WEBHOOK_URL.

Part 9 — CI/CD, Fusion pinned in GitHub Actions too

.github/workflows/ci-snowflake.yml

```
name: CI – lint, compile, test (Snowflake, dbt Fusion)

on:
  pull_request:
    branches: [main]
    paths:
      - 'snowflake/**'

jobs:
  ci:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: snowflake

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'
          cache-dependency-path: snowflake/requirements.txt

      - name: Install pinned Python dependencies
        run: pip install -r requirements.txt

      - name: Install dbt Fusion (pinned to FUSION_VERSION.txt)
        run: |
          curl -fsSL https://public.cdn.getdbt.com/fs/install/install.sh | sh -s -- --update
          echo "$HOME/.local/bin" >> "$GITHUB_PATH"

      - name: Pin Fusion to the exact version and verify
        run: |
          export PATH="$HOME/.local/bin:$PATH"
          FUSION_VERSION=$(cat FUSION_VERSION.txt)
          dbt system update --version "$FUSION_VERSION"
          python scripts/verify_versions.py

      - name: Run pre-commit checks
        run: pip install pre-commit && pre-commit run --all-files

      - name: Run pytest
        run: pytest tests/ -v

      - name: dbt deps
        working-directory: snowflake/dbt
        run: dbt deps

      - name: dbt compile (syntax check only, no warehouse credits used)
        working-directory: snowflake/dbt
        env:
          SF_ACCOUNT: ${ secrets.SF_ACCOUNT }
          SF_USER: ${ secrets.SF_USER }
          SF_PASSWORD: ${ secrets.SF_PASSWORD }
        run: dbt compile --profiles-dir .

      - name: SQLFluff lint
        working-directory: snowflake
        run: |
          sqlfluff lint dbt/models/ \
            --dialect snowflake \
            --templater dbt \
            --format github-annotation
```

.github/workflows/cd-snowflake.yml

```
name: CD – deploy dbt models (Snowflake, dbt Fusion)

on:
  push:
    branches: [main]
    paths:
      - 'snowflake/**'

jobs:
  deploy:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: snowflake

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'
          cache-dependency-path: snowflake/requirements.txt

      - name: Install pinned Python dependencies
        run: pip install -r requirements.txt

      - name: Install and pin dbt Fusion
        run: |
          curl -fsSL https://public.cdn.getdbt.com/fs/install/install.sh | sh -s -- --update
          echo "$HOME/.local/bin" >> "$GITHUB_PATH"
          export PATH="$HOME/.local/bin:$PATH"
          FUSION_VERSION=$(cat FUSION_VERSION.txt)
          dbt system update --version "$FUSION_VERSION"
          python scripts/verify_versions.py

      - name: dbt deps + run + test + snapshot
        env:
          SF_ACCOUNT: ${ secrets.SF_ACCOUNT }
          SF_USER: ${ secrets.SF_USER }
          SF_PASSWORD: ${ secrets.SF_PASSWORD }
        run: |
          cd dbt
          dbt deps --profiles-dir .
          dbt run --profiles-dir . --target prod
          dbt test --profiles-dir . --target prod
          dbt snapshot --profiles-dir . --target prod
          dbt docs generate --profiles-dir . --target prod

      - name: Notify via n8n (Gmail) on success
        if: success()
        run: |
          curl -X POST "${ secrets.N8N_WEBHOOK_URL }" \
            -H "Content-Type: application/json" \
            -d '{"status":"success","dag_id":"github-actions-cd-snowflake","month":"n/a","run_id":"${{ github.sha }}","timestamp":"'$(date -u +%Y-%m-%dT%H:%M:%SZ)'"}'

      - name: Notify via n8n (Gmail) on failure
        if: failure()
        run: |
          curl -X POST "${ secrets.N8N_WEBHOOK_URL }" \
            -H "Content-Type: application/json" \
            -d '{"status":"failure","dag_id":"github-actions-cd-snowflake","month":"n/a","run_id":"${{ github.sha }}","timestamp":"'$(date -u +%Y-%m-%dT%H:%M:%SZ)'"}'
```

.pre-commit-config.yaml — includes the version guard hook

```
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
    - id: check-added-large-files
      args: ['--maxkb=500']
    - id: detect-private-key

- repo: https://github.com/psf/black
  rev: 24.8.0
  hooks:
    - id: black
      language_version: python3.11

- repo: local
  hooks:
    - id: verify-dbt-versions
      name: Verify dbt versions match pinned requirements
      entry: bash -c 'for d in snowflake databricks; do if [ -d "$d" ]; then (cd "$d" && python
scripts/verify_versions.py) || exit 1; fi; done'
      language: system
      pass_filenames: false
      always_run: true
```

1

Install pre-commit

Runs verify_versions on every commit

```
pip install pre-commit
pre-commit install
```

GitHub secret	Used by
SF_ACCOUNT / SF_USER / SF_PASSWORD	ci-snowflake.yml, cd-snowflake.yml
GEMINI_API_KEY	AI agent, if run in CI
N8N_WEBHOOK_URL	cd-snowflake.yml (Gmail alerts on deploy)

Part 10 — Tableau and GitHub publishing

Tableau Public

- Tableau Desktop → Server → Tableau Public → Save to Tableau Public As (not File → Save)
- Sign in with your free Tableau Public account
- Name it "NYC TLC Taxi Pipeline Dashboard", tick Show sheets as tabs
- Get the shareable Link and Embed Code from the Share icon on the published viz

GitHub

```
cd tlc_fusion
git init
git add .
git commit -m "feat: dbt Fusion pipeline, version-locked"
git remote add origin https://github.com/YOUR_USERNAME/nyc-tlc-data-pipeline.git
git branch -M main
git push -u origin main
```

Before the first push, confirm .env is excluded: `cat .gitignore | grep .env` — it must appear for both snowflake/.env and databricks/.env.

- Add CI/CD status badges to the top of README.md
- Set repo description, website (your Tableau URL), and topics via the gear icon
- Pin the repo on your GitHub profile — Customize your pins

Part 11 — Upgrading Fusion later

- Check the Fusion releases page for the new version string
- Update FUSION_VERSION.txt in both snowflake/ and databricks/
- Run: `dbt system update --version <new-version>` locally
- Rebuild Docker: `docker-compose up -d --build`
- Push — CI/CD reads the same file, so it picks up the new pin automatically

Part 12 — Quick reference

Full command sequence, start to finish

```
curl -fsSL https://public.dn.getdbt.com/fs/install/install.sh | sh -s -- --update
dbt system update --version $(cat tlc_fusion/snowflake/FUSION_VERSION.txt)
cd tlc_fusion/snowflake
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python scripts/verify_versions.py
cp .env.example .env # fill in credentials
# run setup_snowflake.sql in Snowflake worksheet
python scripts/load_to_snowflake.py --month 2024-01 --load-zones
cd dbt
dbt deps --profiles-dir .
dbt run --profiles-dir .
dbt test --profiles-dir .
dbt snapshot --profiles-dir .
cd ..
python agent/pipeline_agent.py --warehouse snowflake --month 2024-02
docker-compose up -d --build
```

Project file structure

```
snowflake/
├── .env.example, requirements.txt, FUSION_VERSION.txt
├── Dockerfile.airflow, docker-compose.yml
├── scripts/
│   ├── verify_versions.py
│   ├── load_to_snowflake.py
│   └── setup_snowflake.sql
├── airflow/dags/tlc_snowflake_dag.py
├── dbt/
│   ├── dbt_project.yml, profiles.yml, packages.yml
│   ├── models/{staging,marts,ai}/, schema.yml
│   ├── snapshots/zone_snapshot.sql
│   └── tests/assert_no_future_trips.sql
├── agent/pipeline_agent.py
├── n8n/n8n_gmail_workflow.json
└── tests/test_loader.py
```